ESP Flash Tool

Browser-Based ESP Programmer

User Manual

Black Sphere Industries

Version 1.0 — March 2026

Table of Contents

1 Introduction . . . 4
1.1 Key Capabilities . . . 4
2 Getting Started . . . 4
2.1 System Requirements . . . 4
2.2 Launching the ESP Flash Tool . . . 4
2.3 First-Time Setup . . . 4
3 Features . . . 4
3.1 Flash Tab . . . 4
3.1.1 Connection Section . . . 4
3.1.2 Firmware Files Section . . . 5
3.1.3 Flash Options Section . . . 5
3.1.4 Output Log Section . . . 5
3.2 Monitor Tab . . . 5
3.2.1 Serial Monitor Controls . . . 5
3.2.2 Monitor Terminal . . . 5
3.2.3 Send Bar . . . 5
3.2.4 Monitor Statistics . . . 6
3.3 PWA Installation . . . 6
4 User Interface . . . 6
4.1 Header . . . 6
4.2 Tab Bar . . . 6
4.3 Collapsible Sections . . . 6
4.4 Toast Notifications . . . 6
4.5 Alert Banners . . . 6
5 Workflows . . . 6
5.1 Firmware Flashing Workflow . . . 6
5.2 Flash Erase Workflow . . . 7
5.3 Serial Monitoring Workflow . . . 7
5.4 Debug Workflow . . . 7
6 Architecture . . . 7
6.1 Architecture Overview . . . 8
6.2 Class Diagram . . . 9
6.3 Main Flash Sequence . . . 11
6.4 Monitor and Secondary Operations Sequence . . . 13
6.5 State Diagram . . . 15
7 Configuration . . . 17
7.1 Baud Rate Selection . . . 17
7.2 Flash Mode . . . 17
7.3 Flash Size . . . 17
7.4 Theme . . . 17
8 Troubleshooting . . . 17
8.1 Browser Not Supported . . . 17
8.2 Cannot Connect to Device . . . 17
8.3 Flash Fails Midway . . . 17
8.4 Device Not Entering Bootloader . . . 17
8.5 Monitor Shows Garbage Characters . . . 18

8.6 Erase Takes Too Long 18

1 Introduction

The ESP Flash Tool is a browser-based firmware programmer for Espressif ESP32 and ESP8266 microcontrollers, developed by Black Sphere Industries. It uses the Web Serial API to communicate directly with ESP chips from the browser — no desktop software or drivers required. The tool supports firmware flashing, flash memory erasing, and serial monitoring, all in a single PWA.

1.1 Key Capabilities

- **Firmware flashing** — program ESP32/ESP8266 devices directly from the browser
- **Flash erase** — complete flash memory erase
- **Serial monitor** — real-time serial terminal with configurable baud rates
- **Multi-file support** — flash multiple firmware partitions at specified addresses
- **Chip detection** — automatic identification of connected ESP chip type
- **Flash options** — configurable flash mode (DIO/QIO/DOUT/QOUT), frequency, and size
- **Output logging** — xterm.js-based terminal for flash progress and debug output
- **PWA support** — installable as a standalone application
- **Dark/Light theme** — dual theme support with BSI branding

2 Getting Started

2.1 System Requirements

- **Browser**: Google Chrome 89+, Microsoft Edge 89+, or Opera (Web Serial API required)
- **Operating System**: Windows, macOS, or Linux with USB serial driver support
- **Hardware**: ESP32 or ESP8266 board connected via USB
- Firefox and Safari do not support the Web Serial API and will display an "unsupported browser" message

2.2 Launching the ESP Flash Tool

1. **BSI Portal** — select ESP Flash Tool from the app grid at `blacksphereindustries.nl`
2. **Direct URL** — navigate to the ESP Flash Tool URL
3. **Installed PWA** — launch from your OS app launcher after installation

2.3 First-Time Setup

1. Connect your ESP board to the computer via USB
2. Open the ESP Flash Tool in a supported browser
3. The tool will prompt you to grant serial port access when you click "Connect"

3 Features

3.1 Flash Tab

The Flash tab is the primary interface for firmware programming and consists of four collapsible sections:

3.1.1 Connection Section

- **Baud Rate** — select the communication speed: 921600, 460800, 230400, or 115200 (default)
- **Debug checkbox** — enable verbose logging for troubleshooting
- **Connect button** — open the browser's serial port picker and establish a connection to the ESP chip
- **Disconnect button** — close the serial connection
- **Copy Trace button** — copy the raw serial communication trace to clipboard for debugging

- **Chip Info badge** — after connection, displays detected chip type and MAC address

3.1.2 Firmware Files Section

- **File list** — add one or more firmware binary files, each with:
  - **Address field** — hex memory address for the binary (e.g., `0x0`, `0x1000`, `0x10000`)
  - **File picker** — select a `.bin` firmware file from your computer
  - **Remove button** — remove a file entry
- **Add File button** — add additional firmware file entries for multi-partition flashing
- **Progress bars** — per-file progress indicators during flashing

3.1.3 Flash Options Section

- **Mode** — flash mode selection:
  - `keep` (default) — preserve existing mode
  - `DIO` — Dual I/O
  - `QIO` — Quad I/O
  - `DOUT` — Dual Output
  - `QOUT` — Quad Output
- **Frequency** — flash clock frequency (populated after chip detection)
- **Size** — flash memory size: `detect` (auto-detect) or `keep` (preserve)
- **Flash button** — start the firmware programming process
- **Erase button** — erase the entire flash memory

3.1.4 Output Log Section

An xterm.js-based terminal that displays:

- Connection status messages
- Chip detection results
- Flash progress (percentage per file)
- Error messages and debug output

3.2 Monitor Tab

The Monitor tab provides a full serial terminal for communicating with the ESP device after flashing:

3.2.1 Serial Monitor Controls

- **Baud Rate** — configurable from 9600 to 921600 (default: 115200)
- **Start button** — begin serial monitoring
- **Stop button** — end the monitoring session
- **Reset button** — send a hardware reset signal to the ESP device
- **Clear button** — clear the terminal output

3.2.2 Monitor Terminal

An xterm.js terminal window displaying incoming serial data from the ESP device in real time.

3.2.3 Send Bar

- **Input field** — type commands or data to send to the device
- **Line ending selector** — choose the line ending format:
  - `\\r\\n` (CR+LF, default)
  - `\\n` (LF only)
  - `\\r` (CR only)
  - None

- **Send button** — transmit the input to the device

3.2.4 Monitor Statistics

Three stat cards showing real-time metrics:

- **RX** — bytes received
- **TX** — bytes transmitted
- **Uptime** — connection duration

3.3 PWA Installation

The ESP Flash Tool supports PWA installation:

- Click the "Install" button in the header (appears when the browser supports installation)
- The app will be added to your desktop or home screen
- Runs in a standalone window without browser chrome

4 User Interface

4.1 Header

The header bar displays:

- ESP Flash Tool branding with chip icon
- Connection status indicator (colored dot: green = connected, yellow = connecting, red = disconnected)
- Status text showing current connection state
- Install button (when available)

4.2 Tab Bar

Two tabs below the header:

- **Flash** — firmware programming interface
- **Monitor** — serial terminal interface

Tabs can also be switched via URL hash (`#flash`, `#monitor`).

4.3 Collapsible Sections

Each section in the Flash tab can be collapsed or expanded by clicking the section header. The chevron icon rotates to indicate the current state.

4.4 Toast Notifications

Success and error messages appear as temporary toast notifications at the bottom of the screen.

4.5 Alert Banners

Critical errors (e.g., connection failures) display as persistent alert banners at the top of the Flash tab with a dismiss button.

5 Workflows

5.1 Firmware Flashing Workflow

1. Select the appropriate baud rate (115200 is a safe default; 921600 is faster)
2. Click **Connect** and select the serial port from the browser dialog
3. Wait for chip detection — the Chip Info badge will display the chip type
4. Add firmware file entries with the correct flash addresses:
   - For ESP32: bootloader at `0x1000`, partition table at `0x8000`, application at `0x10000`

- For ESP8266: typically a single binary at `0x0`

5. Select flash options (Mode, Frequency, Size) — use `keep/detect` if unsure
6. Click **Flash** to begin programming
7. Monitor progress in the Output Log terminal
8. After completion, click **Disconnect** or switch to the Monitor tab

5.2 Flash Erase Workflow

1. Connect to the ESP device (same as above)
2. Click **Erase** in the Flash Options section
3. Wait for the erase operation to complete (shown in Output Log)
4. The device's flash memory is now blank and ready for fresh firmware

5.3 Serial Monitoring Workflow

1. Switch to the **Monitor** tab
2. Set the baud rate to match your firmware's serial output rate
3. Click **Start** to begin monitoring
4. View incoming serial data in the terminal
5. Use the Send bar to transmit commands to the device
6. Click **Reset** to reboot the ESP device
7. Click **Stop** to end the monitoring session

5.4 Debug Workflow

1. Enable the **Debug** checkbox before connecting
2. Connect to the device
3. Perform the flash or monitor operation
4. If issues occur, click **Copy Trace** to capture the raw serial communication
5. Share the trace for troubleshooting

6 Architecture

The ESP Flash Tool is a single-file HTML application using vanilla JavaScript, the Web Serial API, and the esptool.js library for ESP communication.

6.1 Architecture Overview

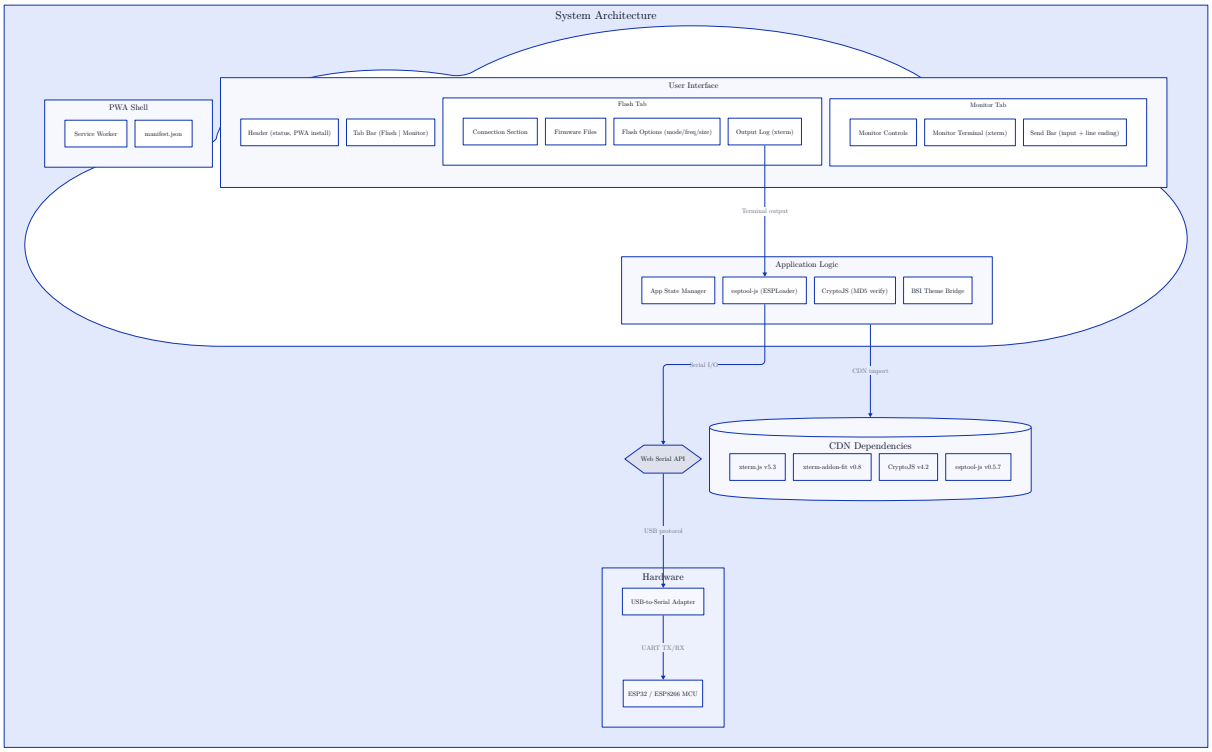

Figure 1: ESP Flash Tool system architecture

6.2 Class Diagram

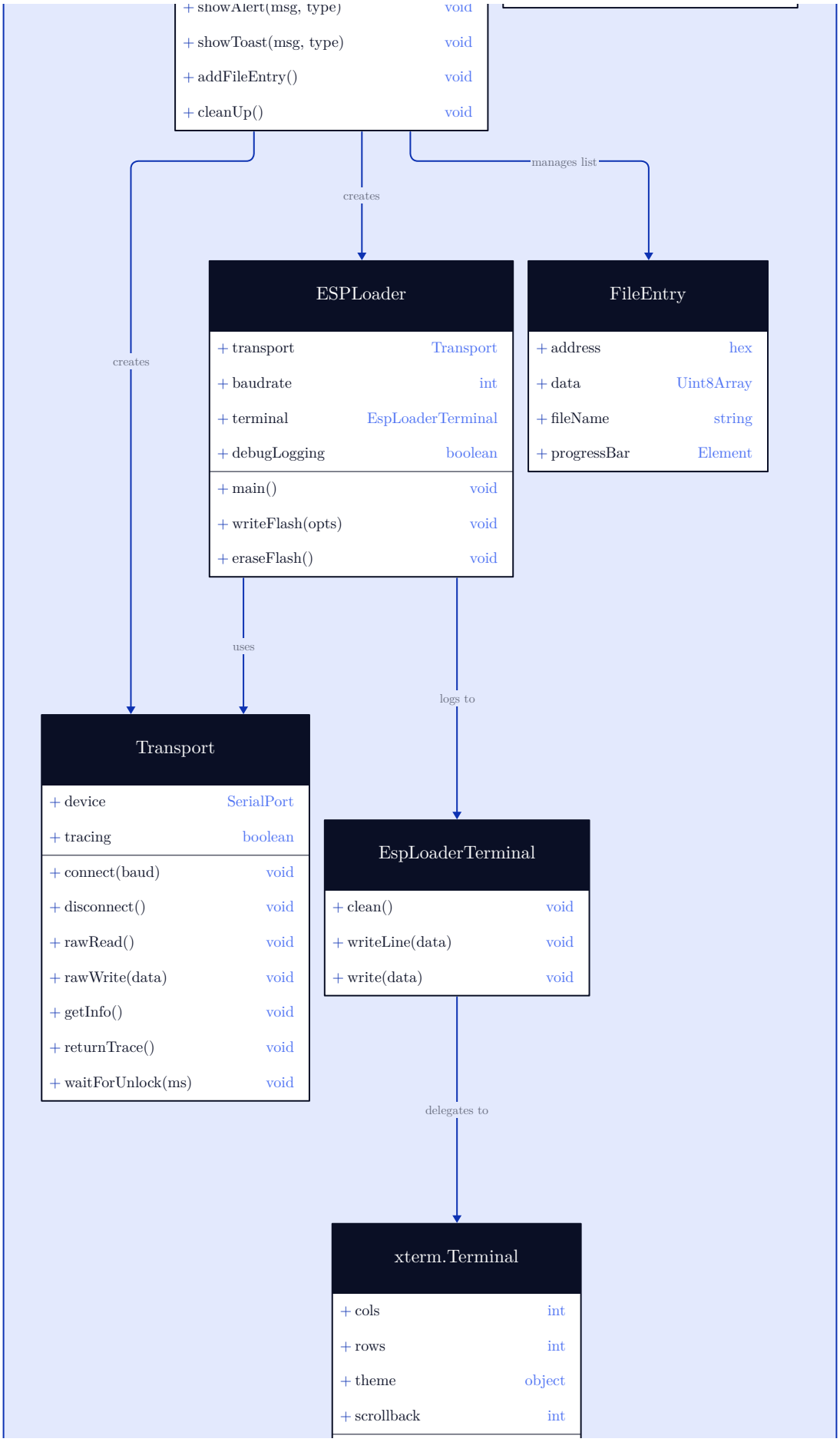

Figure 2: ESP Flash Tool class and component diagram

6.3 Main Flash Sequence

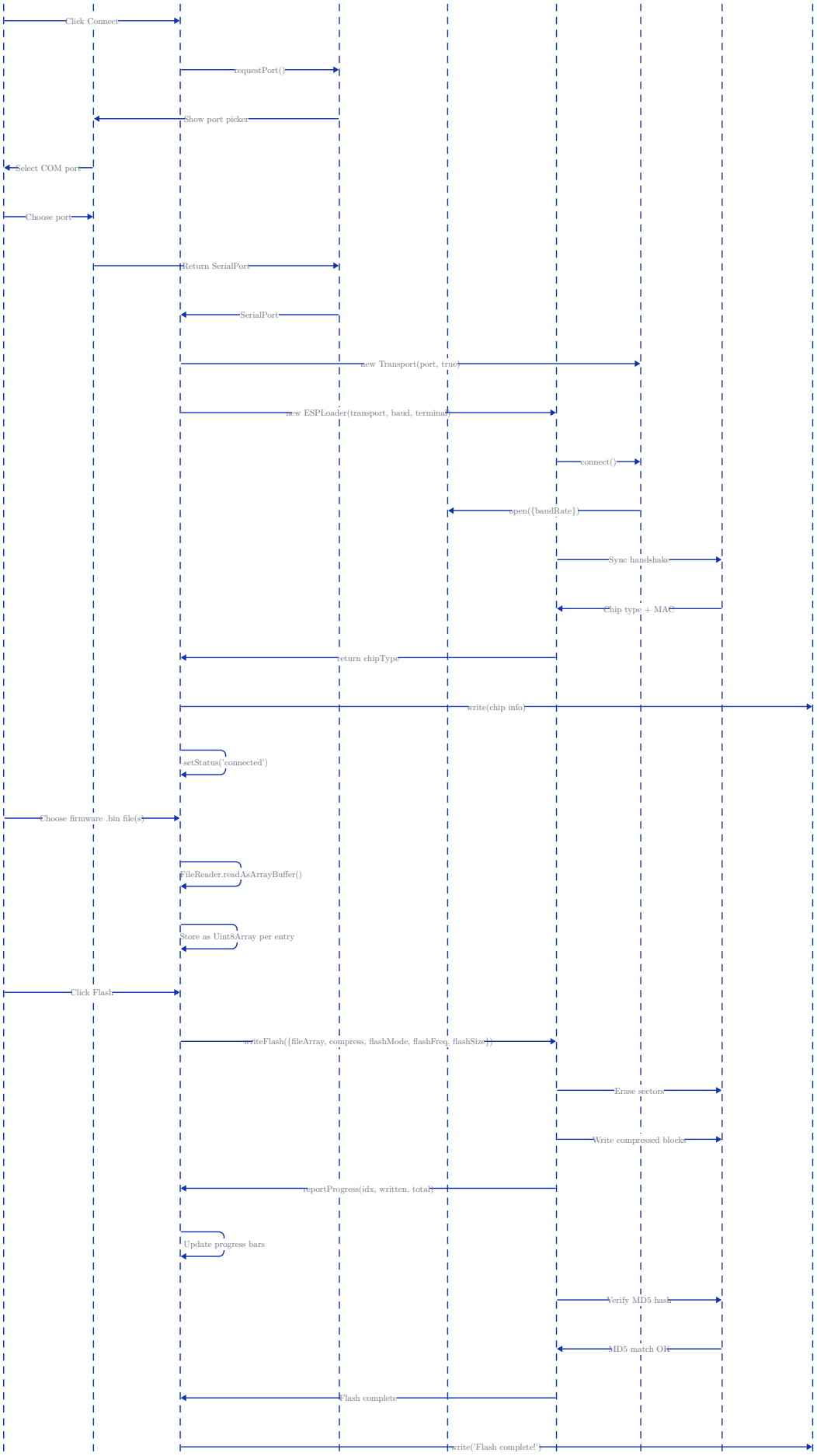

Figure 3: Main firmware flashing sequence

6.4 Monitor and Secondary Operations Sequence

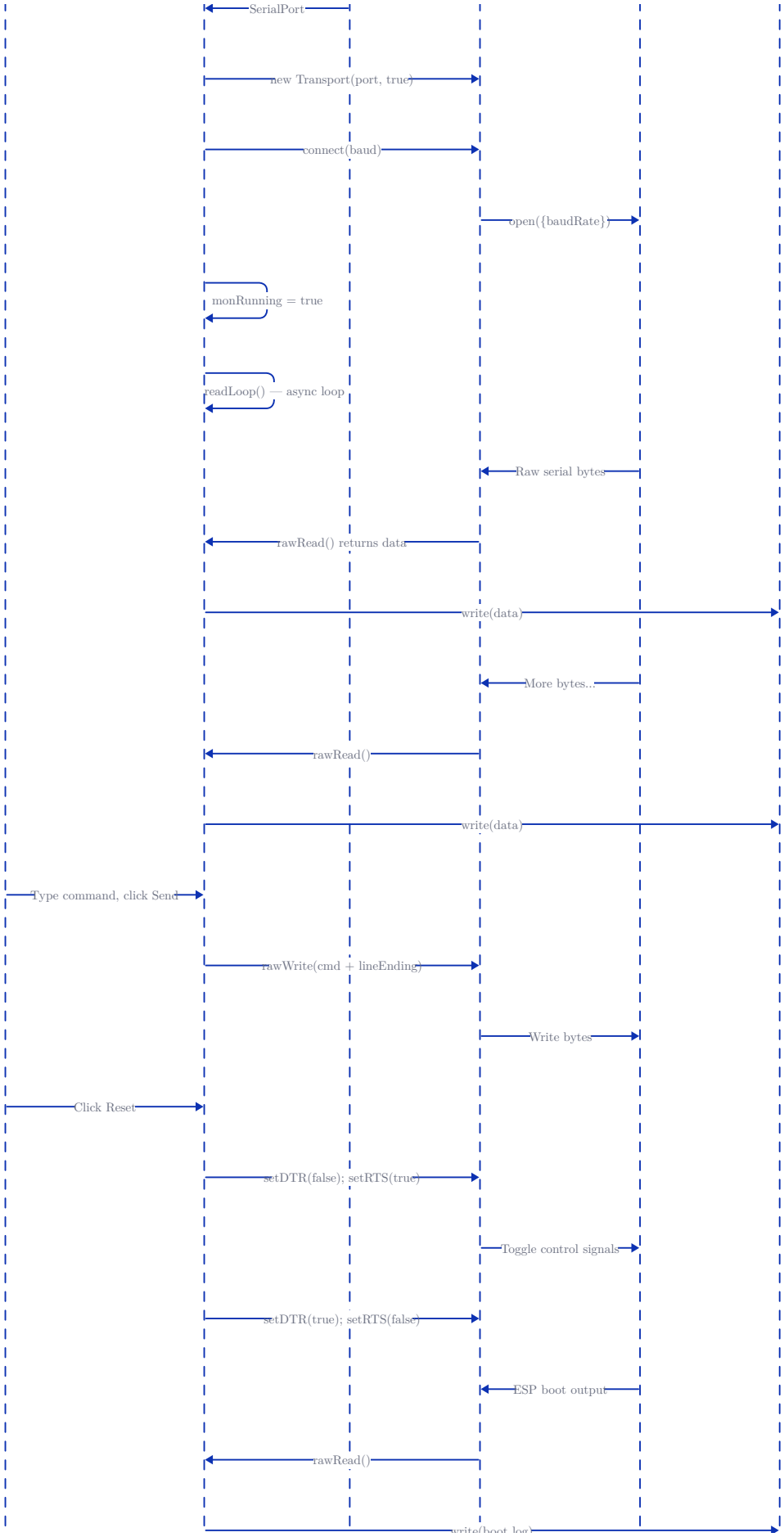

Figure 4: Serial monitor and secondary operations sequence

6.5 State Diagram

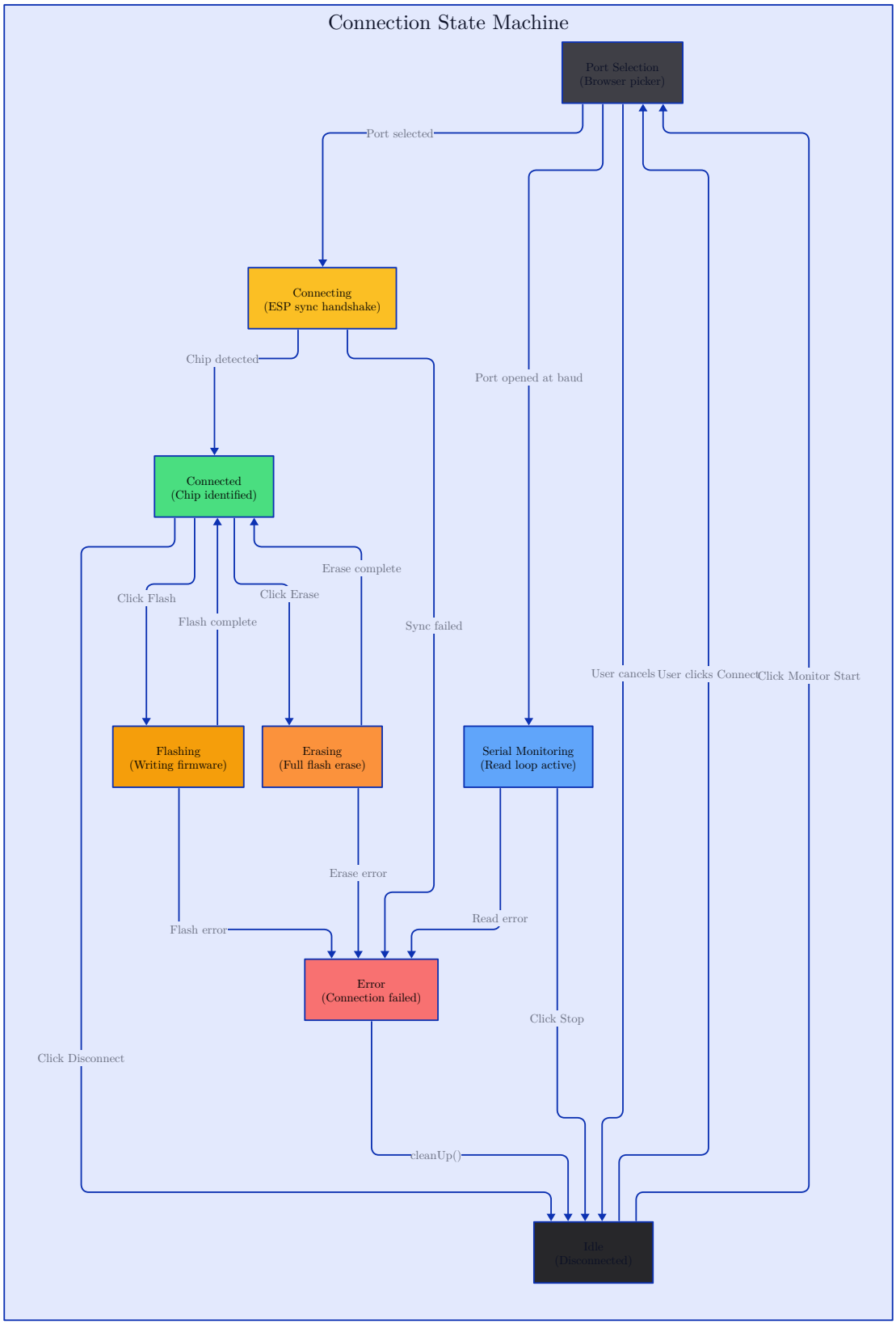

Figure 5: ESP Flash Tool application state diagram

7 Configuration

7.1 Baud Rate Selection

- **115200** — most reliable, works with all ESP devices
- **921600** — fastest, recommended for ESP32 with stable USB connections
- **460800 / 230400** — intermediate options for unstable connections

7.2 Flash Mode

- **DIO** — most compatible, works with all ESP32 modules
- **QIO** — faster, requires quad-SPI flash support
- **DOUT / QOUT** — alternative modes for specific hardware configurations
- **keep** — preserve the existing flash mode (safest option)

7.3 Flash Size

- **detect** — automatically detect flash size from the chip
- **keep** — preserve the existing flash size setting
- Manual sizes: 256KB through 128MB

7.4 Theme

- **Dark** (default) — amber accent on dark background
- **Light** — blue accent on white background
- Theme is controlled by the portal when embedded, or toggled locally in standalone mode

8 Troubleshooting

8.1 Browser Not Supported

- **Symptom**: "Browser Not Supported" message displayed
- **Cause**: Web Serial API is not available
- **Fix**: use Google Chrome, Microsoft Edge (v89+), or Opera

8.2 Cannot Connect to Device

- **Symptom**: no serial port appears in the browser dialog
- **Cause**: USB driver not installed, or device not recognized
- **Fix**: install the appropriate USB-to-UART driver (CP2102, CH340, or FTDI); try a different USB cable (ensure it is a data cable, not charge-only); check Device Manager (Windows) or `ls /dev/tty*` (macOS/Linux)

8.3 Flash Fails Midway

- **Symptom**: flash progress stops and an error appears
- **Cause**: baud rate too high, unstable connection, or incorrect flash addresses
- **Fix**: reduce baud rate to 115200; ensure a stable USB connection; verify firmware addresses match your partition table

8.4 Device Not Entering Bootloader

- **Symptom**: connection timeout or "Failed to connect" error
- **Cause**: ESP device is not in bootloader mode
- **Fix**: hold the BOOT button while clicking Connect; some boards require holding BOOT and pressing RESET; check that the correct serial port is selected

8.5 Monitor Shows Garbage Characters

- **Symptom**: unreadable output in the serial terminal
- **Cause**: baud rate mismatch between the tool and the firmware
- **Fix**: match the monitor baud rate to the value used in your firmware's `Serial.begin()` call (commonly 115200 or 9600)

8.6 Erase Takes Too Long

- **Symptom**: erase operation runs for several minutes
- **Cause**: large flash size (8MB or more)
- **Fix**: this is normal for large flash chips; wait for completion; do not disconnect during erase